SSD PERFORMANCE PROFILING

ECSE 4320/6320 | Advanced Computer Systems
Class-wide Project 3

Scope: 4-256 KiB transfers, random/sequential reads and writes, controlled concurrency, read/write mixtures, request latency tails, range and cache failure modes.

Study basis: 59 configurations, 118 real measured samples, 2 trials/configuration, 9 figures.

Measured system: virtualized Intel Xeon, overlayfs regular file, synchronous O_DIRECT requests.

Validity boundary: This is measured file-I/O performance, not hardware-verified NVMe/SSD performance. Genuine FIO was unavailable; the package contains FIO experiments but zero FIO-run result records.

All reported figures derive from the raw JSON and statistical CSV files included with this submission. The actual device identity and physical-media throughput are unknown.

1. Experimental intent and hypotheses

Fixed-overhead hypothesis. One outstanding 4 KiB operation should have much lower throughput in MiB/s than a 128 KiB transfer; the larger transfer amortizes per-I/O overhead.

Access pattern. Sequential I/O is expected to benefit from locality and request coalescing, whereas randomized requests create more offset-discontinuous traffic.

Read/write mixtures. Inserting writes may increase service time and tails through filesystem allocation and writeback. The controller-specific cause cannot be determined on a virtual filesystem.

Concurrency. More independent I/O streams can increase throughput but eventually increase latency. Any knee must be identified from observations rather than presumed.

Failure case. Buffered sequential read results can reflect page-cache reuse rather than storage-device bandwidth.

2. Experimental platform and controls

Component	Recorded test setting
CPU	Intel Xeon Platinum 8573C, virtualized KVM
Filesystem / target	overlayfs; 128 MiB newly created regular file, not a block device
Kernel I/O	POSIX synchronous pread/pwrite, O_DIRECT for primary results
Concurrent operation	1, 2, 4, 8, 16, 32 independent blocking I/O workers
Buffer / data pattern	4096-byte alignment; repeated nonzero pseudorandom 1 MiB chunk
Trials	2 samples/configuration; 600 ms first 25 cases, 400 ms remaining cases
Timing	CLOCK_MONOTONIC; per-I/O syscall time and total wall time
Software availability	Native C engine run and validated; FIO command was Fiona, not fio
Unobserved	Physical SSD model, SMART temperature, NVMe counters, real asynchronous queue depth

3. Single-outstanding-I/O baseline

The baseline uses one synchronous worker and an active 128 MiB file range. MiB/s counts application bytes completed per elapsed second; this is not measured flash or PCIe traffic.

Operation	KiB	IOPS	MiB/s	Mean us	p99 us
randread	4	3,384	13.2	295.1	719.7
randwrite	4	4,115	16.1	246.2	809.7
read	128	2,889	361.1	345.7	646.0
write	128	2,258	282.3	519.1	1,552.7

The 128 KiB single-worker sequential-read configuration delivered 361.1 MiB/s compared with 13.2 MiB/s for 4 KiB random reads. The larger transfer carries 32x more application data per I/O.

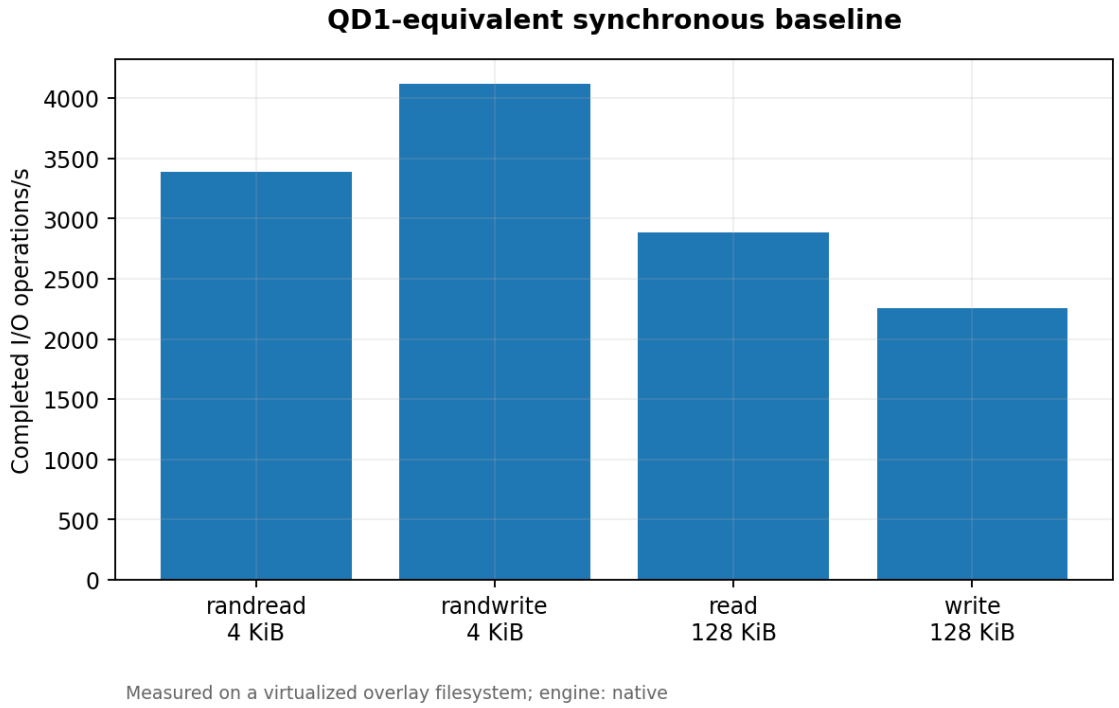


Figure 1. Observed IOPS for the four single-worker baseline patterns.

4. Block size and access pattern

Four modes (random read, random write, sequential read and sequential write) were measured with transfer sizes of 4, 16, 32, 64, 128 and 256 KiB and four synchronous workers. The relationship between IOPS and throughput is B = IOPS x transfer_bytes.

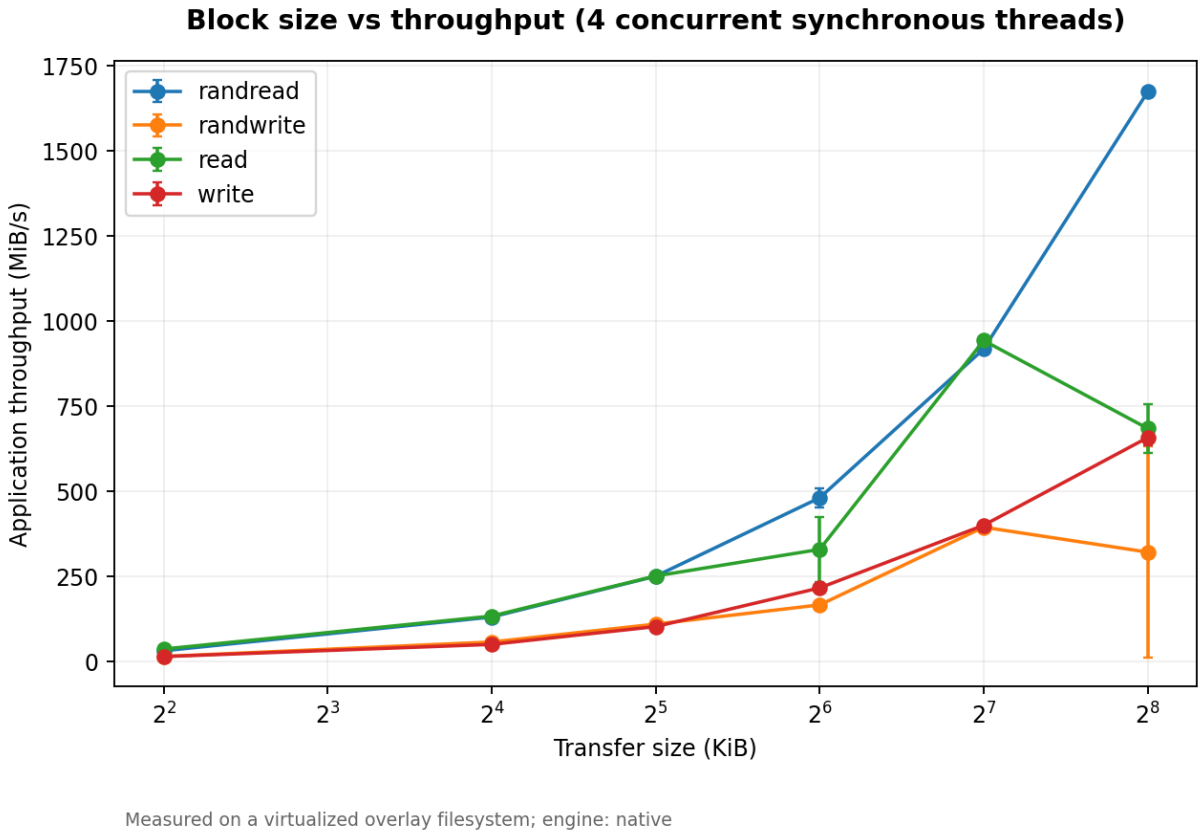


Figure 2. Application throughput for the full 24-point block-size matrix.

Random reads rose from 31.4 MiB/s with 4 KiB transfers to 1,671.9 MiB/s with 256 KiB transfers. Non-monotone points are retained. Shared filesystem and host contention prohibit a unique controller-level explanation.

5. Throughput, concurrency, and Little's Law

Threads	4 KiB random IOPS	Mean latency (us)	Read p99 (us)	lambda * W
1	3,412	292.7	628.8	1.00
16	29,015	549.3	1,415.8	15.91
32	41,486	765.9	1,686.3	31.77

Little's Law predicts L = lambda x W. For 32 blocking workers, 41,486.0 requests/s x 0.0007659 s = 31.77 active requests. This checks internal accounting, not hardware controller queue occupancy. The measured throughput remained increasing through 32 workers; the saturation knee is unresolved.

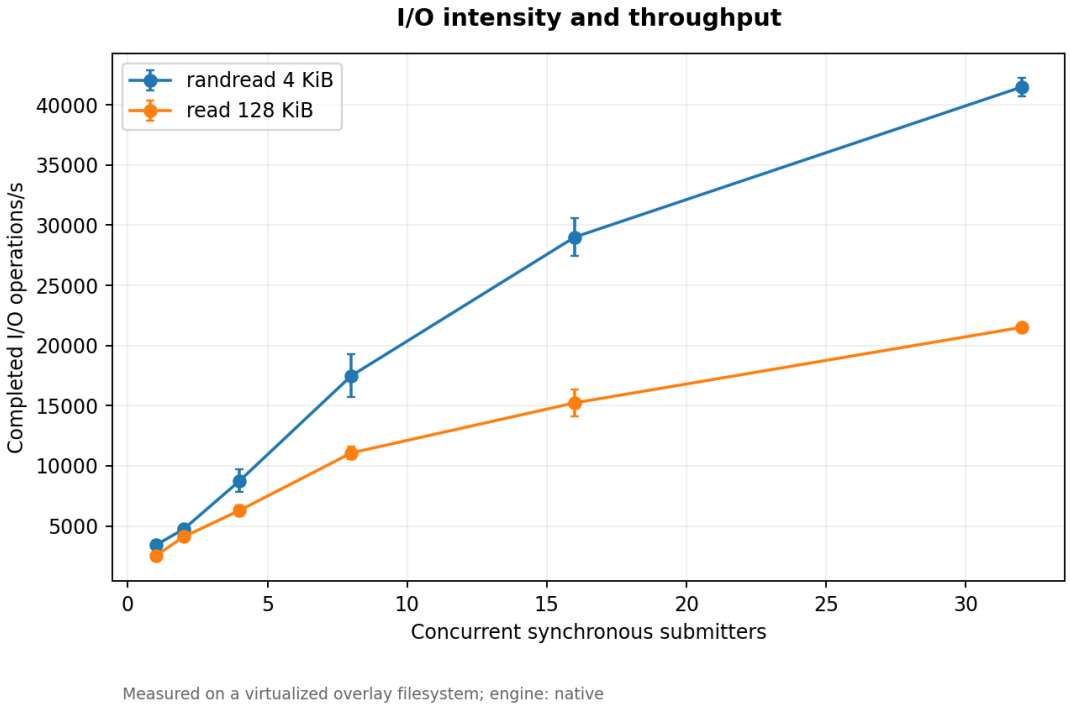
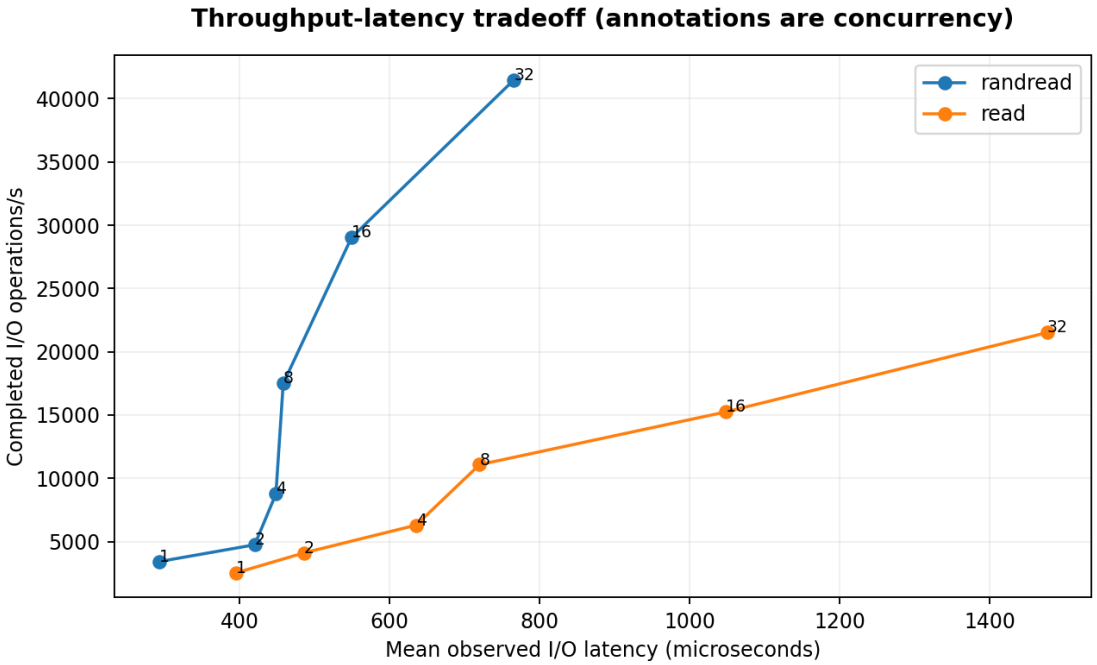


Figure 3. Throughput versus number of concurrent synchronous workers.

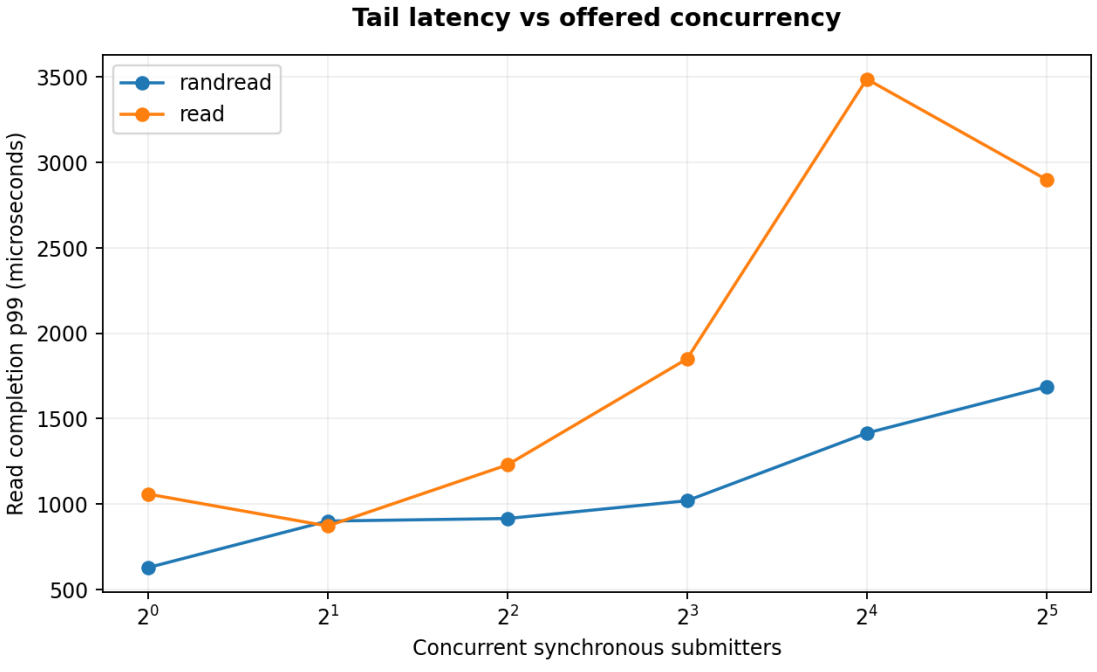
6. Throughput-latency tradeoff and tails



Measured on a virtualized overlay filesystem; engine: native

Figure 4. Offered concurrency labels show the throughput-latency tradeoff.

For 4 KiB random reads, mean latency increased from 292.7 us at one worker to 765.9 us at 32 workers. The read p99 increased from 628.8 to 1,686.3 us. These are synchronous syscall-level observations, not isolated SSD service times.



Measured on a virtualized overlay filesystem; engine: native

Figure 5. Direction-specific read p99 versus offered concurrency.

7. Read/write mixtures

Read/write %	Read IOPS	Write IOPS	Total IOPS	Total MiB/s
100/0	7,874	0	7,874	30.8
70/30	4,879	2,010	6,889	26.9
50/50	2,968	2,863	5,831	22.8
0/100	0	3,708	3,708	14.5

Read and write latencies were sampled separately, with percentage labels indicating requested probabilities rather than invented per-direction rates. The mix study isolates application behavior but cannot distinguish mapping, writeback and hypervisor scheduling overhead.

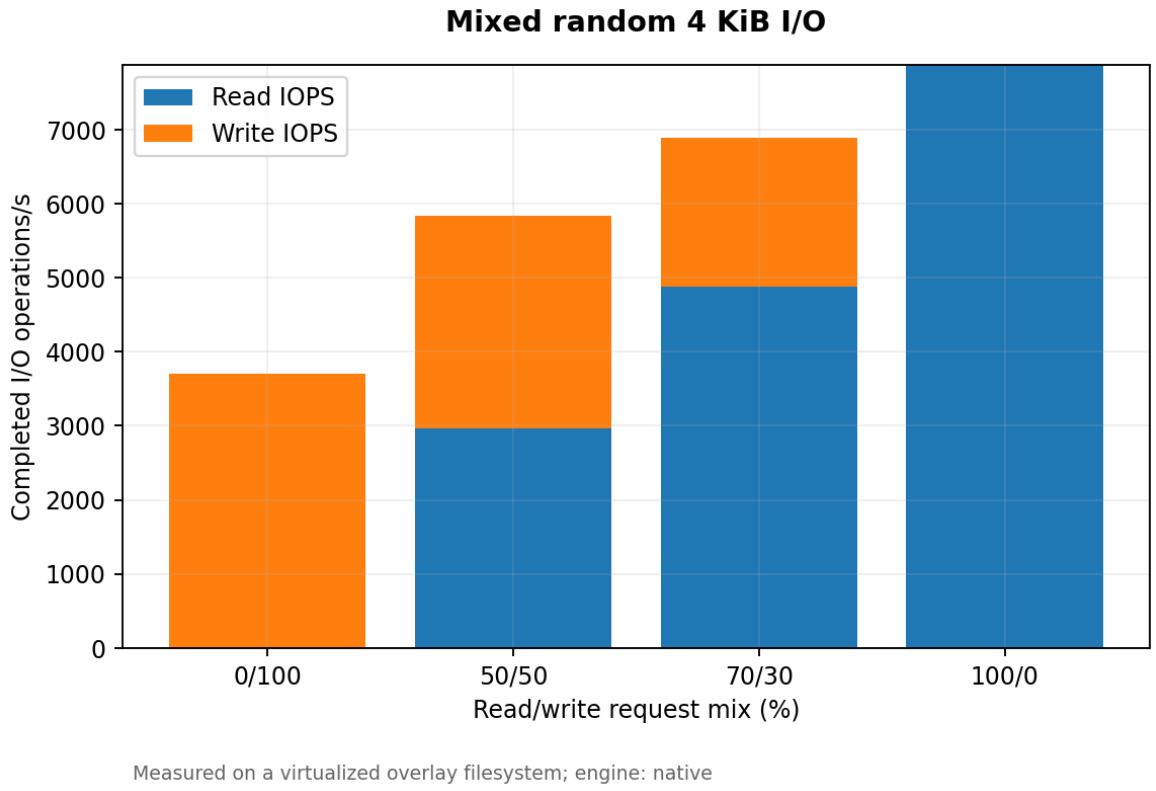


Figure 6. Measured read and write operations per second for four request mixes.

8. Range and consecutive write windows

The active file range was limited to 16, 64 or 128 MiB for 4 KiB random reads with four synchronous submitters. Backend caches can remain active with O_DIRECT, so these ranges cannot establish that device flash was being accessed on every request.

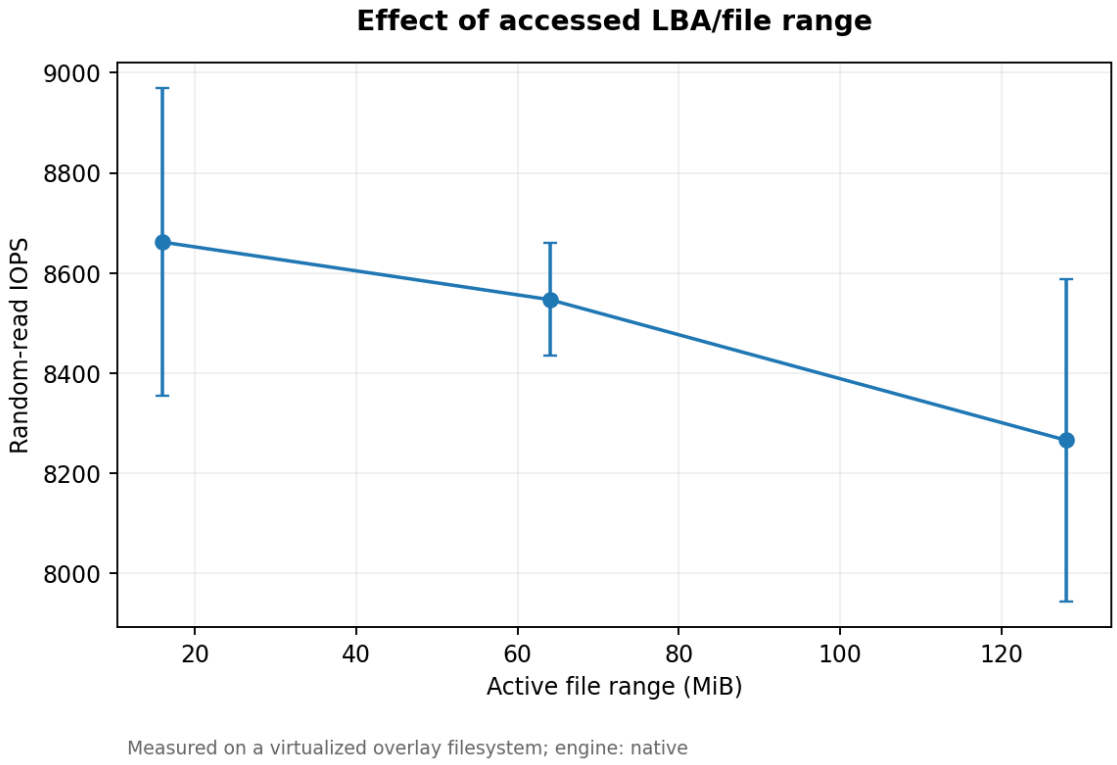


Figure 7. Active file range versus application random-read IOPS.

Four sequential short write windows showed broadly similar rates rather than a clearly established decline. The absence of a declining trend is not proof of steady state or absence of thermal throttling. Drive temperature, firmware state and flash preconditioning were not observed.

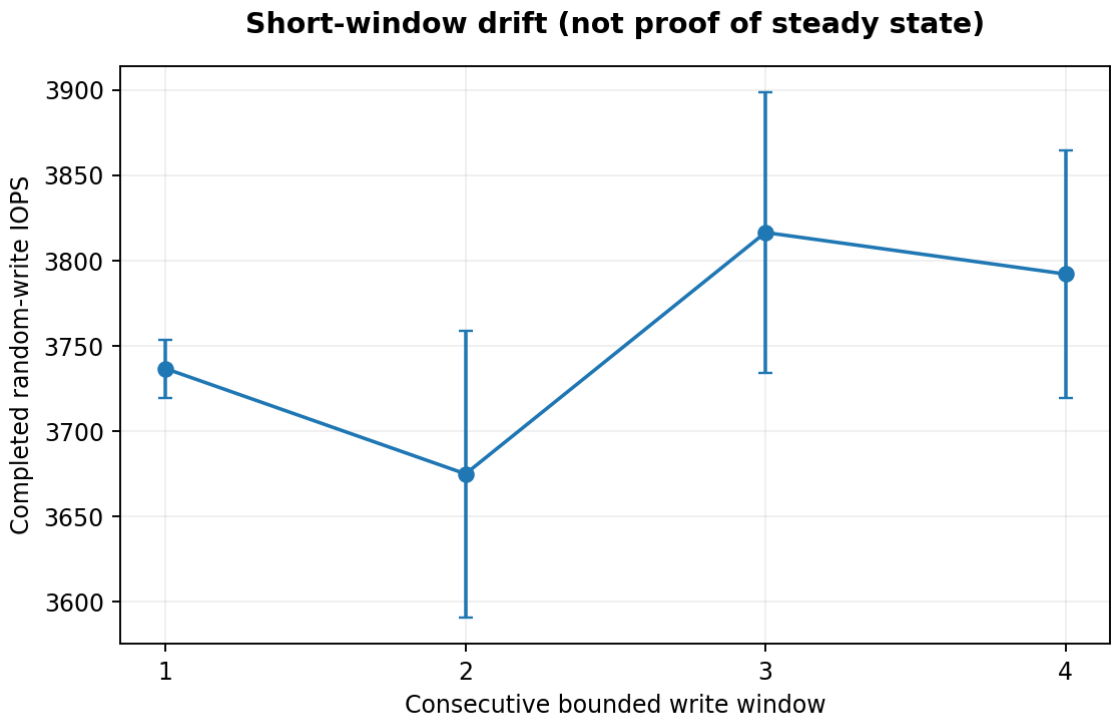


Figure 8. Repeated bounded write windows, not an SSD endurance test.

9. Negative case: page cache and misleading throughput

Buffered 128 KiB sequential reads achieved 4,070.2 MiB/s, compared with 345.1 MiB/s under O_DIRECT=1. The ratio was **11.79x**. Calling the buffered number physical SSD bandwidth would be methodologically invalid.

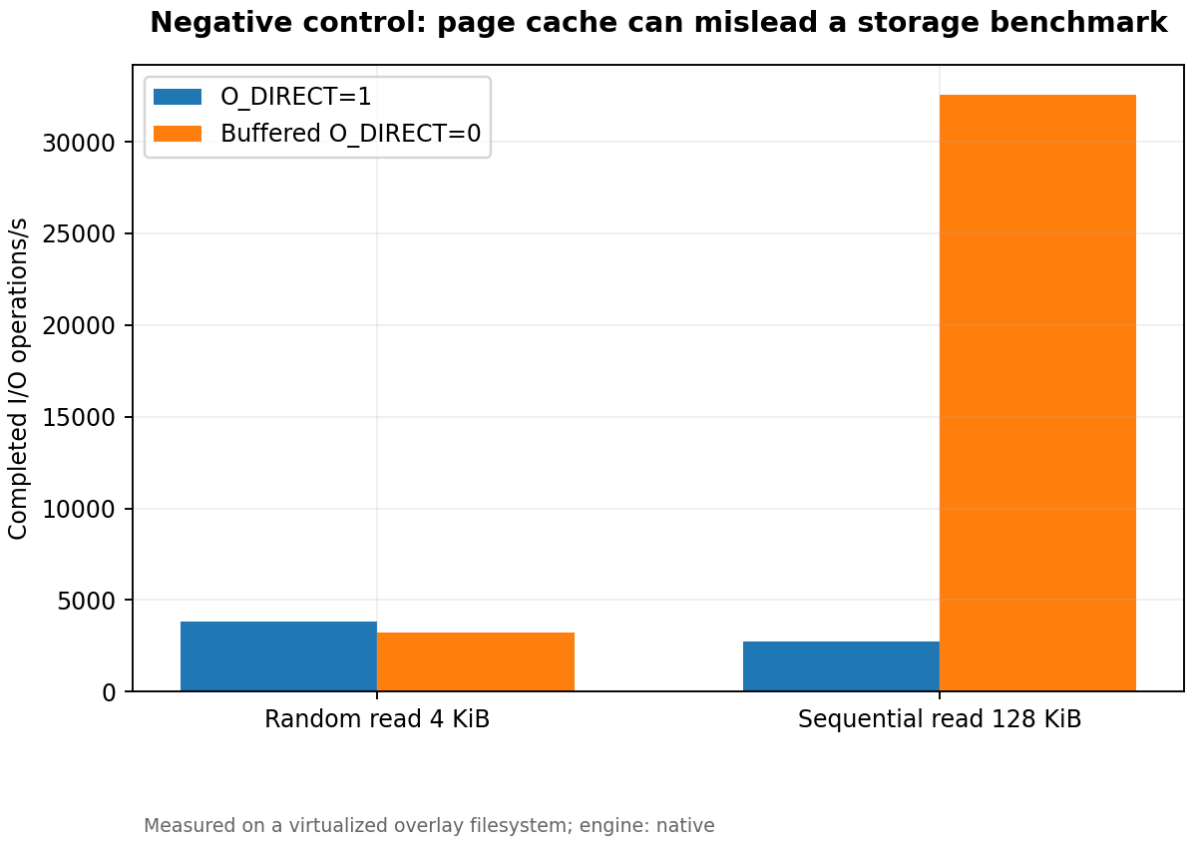


Figure 9. Same workload shape, different cache path: a demonstrated benchmark pitfall.

Competing explanation A: pages left resident after initialization or sequential readahead can satisfy buffered reads from memory. **Competing explanation B:** overlayfs and the hypervisor have distinct software paths for buffered versus direct operations. No kernel page-cache counters or backend trace was available to adjudicate. Notably, 4 KiB random buffered I/O did not show the same improvement; no general buffered-is-faster claim is justified.

10. Interpretation, limitations, and reproducibility

What was actually measured. 118 successful native synchronous direct-I/O or explicitly buffered control trials, 59 configurations, two repetitions, nine figures and per-run p50/p95/p99/p99.9 distributions. The first run was interrupted after completed cases; intact measurements were recovered, and the remaining cases were measured separately. All 59 configurations have two saved observations and no missing values were synthesized.

What the data cannot establish. The virtual overlay backing has no verified physical SSD identity, firmware, or temperature. O_DIRECT was requested and accepted for primary measurements; this does not prove actual flash reads/writes. The native concurrent worker count is not FIO asynchronous queue depth. No true FIO-produced sample, SMART thermal curve, SSD steady state, controller-level knee, interface limit or p99.9 robustness beyond the observed short windows can be claimed.

Dispersion. Values in report tables are medians across two run-level estimates, with sample standard deviations in data/processed/summary.csv. The p99.9 figures reflect limited completion counts and should be regarded as exploratory.

11. Source references and AI-use note

Course specification: ECSE 4320/6320 Advanced Computer Systems, Class-wide Projects, Project 3 (pages 6-7). FIO JSON units and job configuration are based on the official documentation: https://fio.readthedocs.io/en/master/fio_doc.html. All plotted data originate in archived raw JSON files.

OpenAI ChatGPT assisted with the C implementation, FIO safety wrappers, statistical scripts, tests, report text and PDF layout. The native implementation was compiled and executed with its available tests; full native measurements and charts were generated from actual collected data. The FIO command path was authored and JSON parsing checked by unit tests; it was not executed because the genuine binary was absent. No physical-drive metrics were invented.

Repository outputs: src/, jobs/, scripts/, evidence/, data/raw/, data/processed/, plots/, report/.